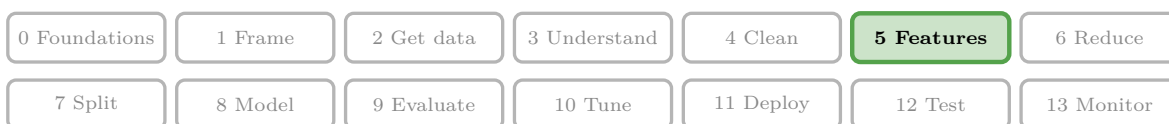


Feature Construction and Feature Splitting

Contents

1. Overview	2
2. Feature construction	2
3. Family type on the Titanic	3
3.1 The data and a baseline score	3
3.2 Building the family size	4
3.3 Grouping the size into a family type	4
3.4 Dropping the old columns and scoring again	5
4. Constructing features in other data	6
5. Feature splitting	7
5.1 Tidy data	7
5.2 Splitting a column	7
6. Splitting the title out of the Titanic names	7
7. What the title tells us	9
7.1 A married column from the title	10
8. Measuring whether each new column helps	10
9. Summary	13
10. Sources	13
11. Key terms	13

Where this fits: Pipeline map step 5 (Engineer features). See the [Course map](#).



- **Builds on:** Feature engineering ([Note 23](#)); Cross-validation ([Note 30](#)); Binning and binarization ([Note 32](#)).
- **Compare with:** Feature extraction ([Note 23](#)).

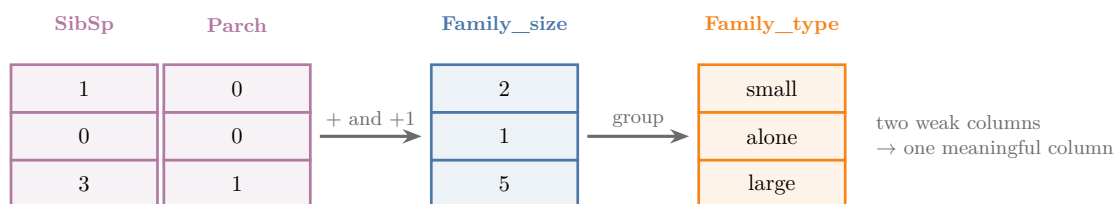
1. Overview

Key point: Feature construction builds a new column from existing ones by hand; feature splitting breaks one column that holds several facts into several columns.

Every technique in the Notes so far changed a column that was already there: filling gaps, encoding, scaling, removing outliers. Those are all feature transformation. This Note covers the second part of feature engineering: making new columns.

Figure 1 shows the two ideas on the Titanic data. Construction combines **SibSp** and **Parch** into a family type. Splitting pulls the title (Mr, Miss, Master) out of each passenger's name.

Feature construction: build a new column from existing ones



Feature splitting: one cell holds several facts

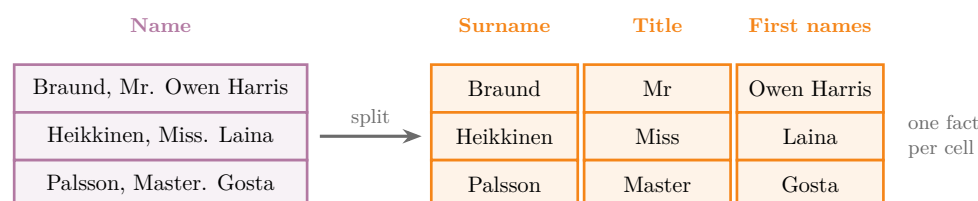


Figure 1: Feature construction (top) and feature splitting (bottom) on the Titanic data

2. Feature construction

Key point: There is no formula for feature construction: the new column comes from our own intuition, domain knowledge and experience.

Every earlier technique had a fixed procedure. Standardization, for example, always subtracts the mean and divides by the standard deviation. Feature construction has no such procedure.

Feature construction means creating a new column by hand, from existing columns, because we believe it will help the model (see the [feature engineering Note](#), section 7). Three things decide which column we build:

- **intuition:** a sense of what could matter for the target;
- **domain knowledge:** knowing the field the data comes from;
- **experience:** after working on many datasets, useful columns become easier to spot.

At first it feels hard, because it is not obvious which columns could be built from which. The habit to build is simple: for every new dataset, ask “can I construct a new column that helps the model?”

3. Family type on the Titanic

Key point: SibSp and Parch add up to a family size, and the family size groups into three family types: alone, small and large.

3.1 The data and a baseline score

Key point: Before building anything, we score a model on the original columns, so that we have a number to compare against.

We use four input columns of the Titanic training data and the target:

Column	Meaning
Age	age in years
Pclass	ticket class: 1, 2 or 3
SibSp	number of siblings and spouses on board
Parch	number of parents and children on board
Survived	1 if the passenger survived, 0 if not (target)

Age is missing for 177 of the 891 passengers. To keep the example simple we drop those rows, which leaves 714.

The baseline is logistic regression scored with 20-fold cross-validation (Notes 29 and 30). It gets a mean accuracy of 69.3%.

Python: The baseline score.

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

cols = ["Age", "Pclass", "SibSp", "Parch", "Survived"]
```

```
df = pd.read_csv("data/titanic.csv")[cols].dropna()
X = df.drop(columns="Survived")
y = df["Survived"]

lr = LogisticRegression(max_iter=1000)
cross_val_score(lr, X, y, cv=20).mean() # 0.693
```

3.2 Building the family size

Key point: Family size = SibSp + Parch + 1, where the 1 counts the passenger.

SibSp and Parch both count relatives on board. Added together, they give the number of relatives. Adding 1 for the passenger gives the **family size**: the number of people travelling together as a family.

1. **In words:** siblings and spouses, plus parents and children, plus the passenger.
2. **Formula:**

$$\text{Family_size} = \text{SibSp} + \text{Parch} + 1$$

3. **Example:** Mr. Owen Harris Braund travelled with one sibling or spouse and no parents or children, so

$$\text{Family_size} = 1 + 0 + 1 = 2.$$

Python: One new column from two old ones.

```
X["Family_size"] = X["SibSp"] + X["Parch"] + 1
```

Adding two columns adds them row by row, so every passenger gets their own family size.

3.3 Grouping the size into a family type

Key point: Family size 1 means alone (0), 2 to 4 means a small family (1), and 5 or more means a large family (2).

Instead of the exact size, we keep only the kind of group the passenger was in. This new column is the **family type**:

Family size	Family type	Code
1	alone	0
2 to 4	small family	1
5 or more	large family	2

Figure 2 shows why these groups make sense. Passengers alone survived 30% of the time, small families 55% to 72%, and large families 0% to 33%.

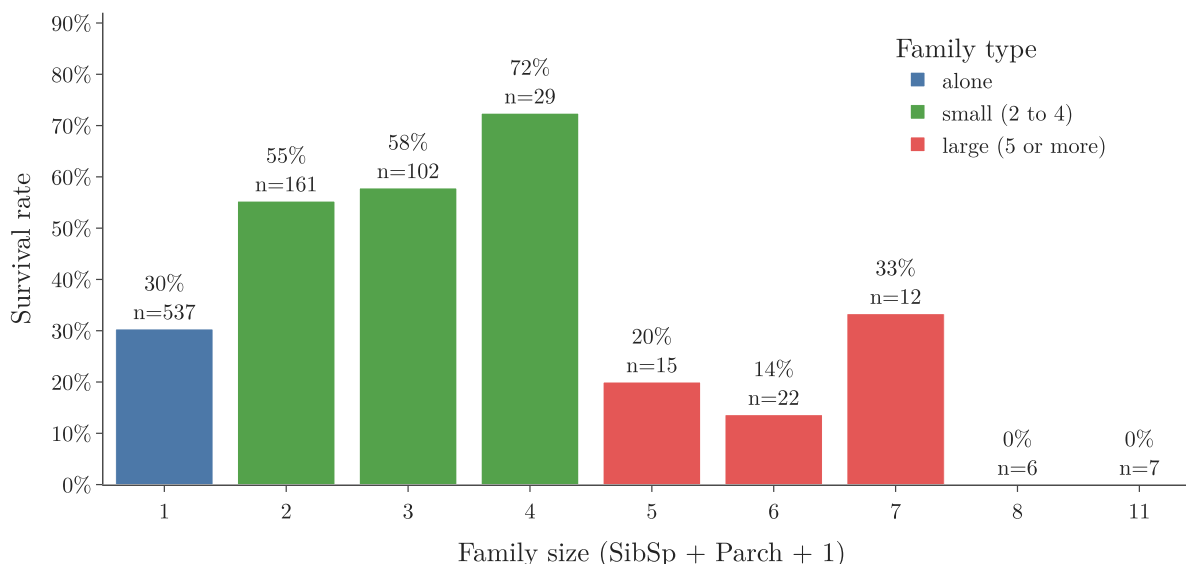


Figure 2: Survival rate for each family size, coloured by family type (all 891 passengers)

The pattern is not a straight line. Survival rises from alone to small families, then falls sharply for large families. A single column that says which of the three groups a passenger belongs to captures this better than the two raw counts.

Python: Grouping with `np.select`.

```
import numpy as np
```

```
size = X["Family_size"]
# first matching condition wins; otherwise 2
X["Family_type"] = np.select(
    [size == 1, size <= 4], [0, 1], 2)
```

`np.select(conditions, values, default)` gives each row the value of the first condition it meets. A size of 3 fails `size == 1`, meets `size <= 4`, and gets 1. Writing a small function and calling `.apply` on the column gives the same result, one row at a time.

3.4 Dropping the old columns and scoring again

Key point: Family type replaces `SibSp`, `Parch` and `Family_size`; the score moves from 69.3% to 70.0%.

`Family_type` now carries what `SibSp`, `Parch` and `Family_size` said, so we drop those three. The model is left with `Age`, `Pclass` and `Family_type`: three columns instead of four.

With the same 20-fold cross-validation, the mean accuracy goes from 69.3% to 70.0%. The model is slightly better with fewer columns.

Python: Dropping the old columns and scoring again.

```
X = X.drop(columns=["SibSp", "Parch", "Family_size"])
cross_val_score(lr, X, y, cv=20).mean() # 0.700
```

Extra: How big is a 0.7-point gain? On 714 passengers it means about 5 more passengers predicted correctly.

Each of the 20 folds holds only about 36 passengers, and the score of a single fold moves by about 9 points from fold to fold. Section 8 checks the gain more carefully, and it stays small.

4. Constructing features in other data

Key point: Domain knowledge suggests the column: in cricket data, strike rate for batters and economy for bowlers.

The same habit works on any dataset. In IPL ball-by-ball data, anyone who follows cricket knows two numbers that rate players better than raw totals.

Strike rate, for a batter:

1. **In words:** runs scored for every 100 balls faced.

2. **Formula:**

$$\text{strike rate} = \frac{\text{runs}}{\text{balls faced}} \times 100$$

3. **Example:** 45 runs off 30 balls give

$$\frac{45}{30} \times 100 = 150.$$

Economy rate, for a bowler:

1. **In words:** runs given away per over bowled.

2. **Formula:**

$$\text{economy} = \frac{\text{runs conceded}}{\text{overs bowled}}$$

3. **Example:** 32 runs in 4 overs give

$$\frac{32}{4} = 8.$$

Neither column exists in raw ball-by-ball data. Both are built from columns that do, using knowledge of the game.

Extra: Strike rate and economy are ratios, and ratios are a natural kind of constructed feature in other data too: price per square foot, debt per unit of income, clicks per visit. The intuition: a ratio removes the effect of size, so a big flat and a small flat can be compared by their price per square foot. Taking the ratio of two columns is one of the first new features worth trying, and the model then tells us whether it helps (Kuhn and Johnson, §1.1).

5. Feature splitting

Key point: In tidy data every cell holds one value; when a cell packs several facts together, we split it into several columns.

5.1 Tidy data

Key point: Tidy data has one observation per row and one atomic value per cell.

Data is **tidy** when:

- each row is one observation (here, one passenger);
- each cell holds an **atomic value**: one single piece of information, not several pushed into one place.

Sometimes a dataset is not tidy: one cell holds two or three facts at once. Such a column is hard to plot, group or give to a model, because the facts inside it cannot be used separately.

Extra: The usual definition of tidy data, from the statistician Hadley Wickham, has three rules (Wickham and Grolemund, §12.2): each variable is a column, each observation is a row, and each cell holds one value. A name column that holds a title and a first name breaks the first and third rules: two variables share one column and one cell.

5.2 Splitting a column

Key point: A name like “Mr. Ankit” holds a title and a first name; splitting gives one column for each.

Take a name column whose value is “Mr. Ankit”. It holds two facts: the **title** Mr (the word before the name that shows sex, and often marital status or rank) and the first name Ankit.

Feature splitting breaks such a column into one column per fact: here, a **Title** column with Mr and a **First_name** column with Ankit. The new columns can affect the model in ways the combined column could not. Splitting is useful in many situations, and Note 33 already split two Titanic columns, **Cabin** and **Ticket**, in the same way.

6. Splitting the title out of the Titanic names

Key point: Split each name at “,” and keep the second piece, then split that at “.” and keep the first piece: the result is the title.

Every name in the Titanic data has the same shape: surname, a comma, the title with a full stop, then the other names. For example, “Braund, Mr. Owen Harris”.

Figure 3 shows how two splits isolate the title:

1. Split at “,” (comma and space). Piece 0 is the surname, Braund; piece 1 is “Mr. Owen Harris”.
2. Split piece 1 at “.”. Piece 0 is the title, Mr.

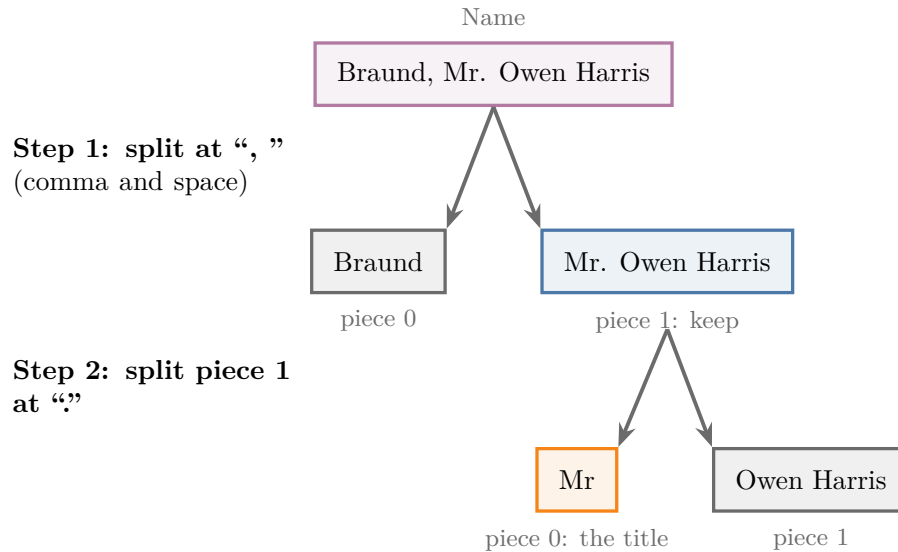


Figure 3: Two splits pull the title out of a name

Python: Splitting the title out of Name.

```
df = pd.read_csv("data/titanic.csv")
after_comma = df["Name"].str.split(", ").str[1]
df["Title"] = after_comma.str.split(".").str[0]
.str.split(", ") turns each name into a list of pieces, and .str[1] takes piece 1 of every list. The second line does the same at the full stop and keeps piece 0.
```

Name	Title
Braund, Mr. Owen Harris	Mr
Cumings, Mrs. John Bradley (Florence Briggs Thayer)	Mrs
Heikkinen, Miss. Laina	Miss
Futrelle, Mrs. Jacques Heath (Lily May Peel)	Mrs
Allen, Mr. William Henry	Mr

The 891 names give 17 different titles. One unusual name, “Roths, the Countess. of (Lucy Noel Martha Dyer-Edwards)”, gives the title “the Countess”.

Extra: Older code splits with `expand=True`, which returns a table, and then picks a column: `df["Name"].str.split(", ", expand=True)[1]`. The `.str[1]` form above gives the same values without building the table.

7. What the title tells us

Key point: Survival rate differs strongly by title: 79% for Mrs, 70% for Miss, 57% for Master, but only 16% for Mr.

Grouping passengers by title and averaging **Survived** gives each title's survival rate (Figure 4):

- **Mrs** (married women): 79% of 125 survived, the highest of the common titles.
- **Miss** (unmarried women and girls): 70% of 182.
- **Master** (boys): 57% of 40.
- **Mr** (men): only 16% of 517.

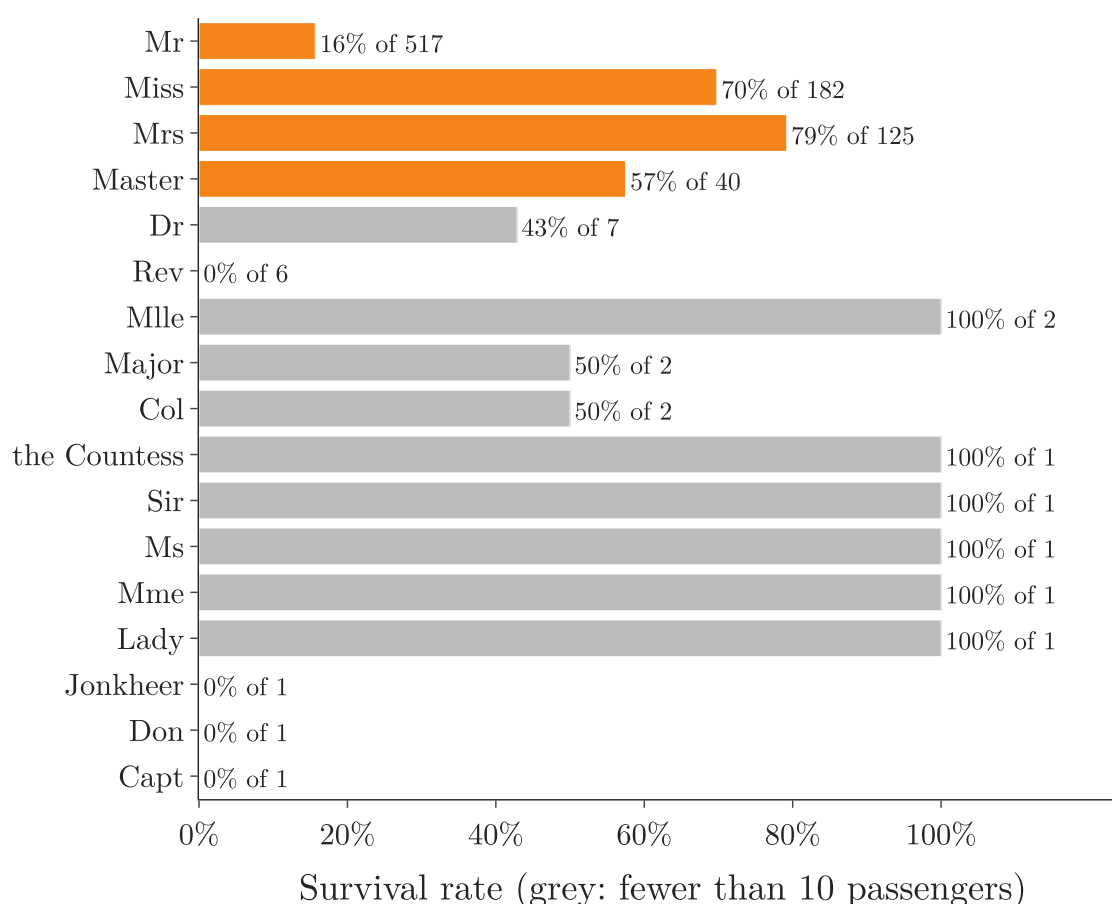


Figure 4: Survival rate for each title; titles with fewer than 10 passengers in grey

The title carries a lot of information about survival that the raw name hid. It is a very good new column for improving the model.

Python: Survival rate by title.

```
rate = df.groupby("Title")["Survived"].agg(["mean", "size"])
```

```
rate.sort_values("size", ascending=False)
agg(["mean", "size"])
```

 gives each title's survival rate and its number of passengers. Select the `Survived` column before averaging: `df.groupby("Title").mean()` fails in pandas 3, because it tries to average text columns such as `Name`.

Extra: Treat the grey bars in Figure 4 with care. Lady, Sir, Ms and the Countess show 100% survival, but each is a single passenger. One person cannot give a reliable rate. So before using the column in a model, we group rare titles into one category. `scikit-learn`'s `OneHotEncoder` can do this for us (Section 8).

7.1 A married column from the title

Key point: The title also gives a new 0/1 column: `Is_Married` is 1 when the title is Mrs.

Splitting and construction often work together. From the split-out title we can construct another column, `Is_Married`: 1 for passengers with the title Mrs, 0 for everyone else.

Python: Building `Is_Married`.

```
df["Is_Married"] = (df["Title"] == "Mrs").astype(int)
df["Is_Married"].sum() # 125
```

`df["Title"] == "Mrs"` gives True or False for every row, and `.astype(int)` turns these into 1 and 0.

Extra: Older code builds the column in two steps: `df["Is_Married"] = 0`, then `df["Is_Married"].loc[df["Title"] == "Mrs"] = 1`. The second line first selects the column and then changes the selection, which is called **chained assignment**. In pandas 3 the selection is always a separate copy, so the line prints a `ChainedAssignmentError` warning and changes nothing: every row stays 0. The one-step version above avoids the problem.

Extra: `Is_Married` only marks married women. Married men are all “Mr”, like unmarried men, so the title cannot tell them apart.

8. Measuring whether each new column helps

Key point: Of all the new columns, the split-out title helps most (69.4% to 81.6%); family size alone adds nothing to a linear model.

Extra: This whole section goes beyond the original example, which scored only the family type. A new column is only worth keeping if the model scores better with it, and a small gain can be pure luck.

We test every column the same way:

- the same 714 passengers with a known age;
- logistic regression;

- 10-fold cross-validation repeated 10 times with different shuffles, so each score is the mean of 100 folds;
- every new column built inside a pipeline (Note 29), so each fold builds it from its own rows.

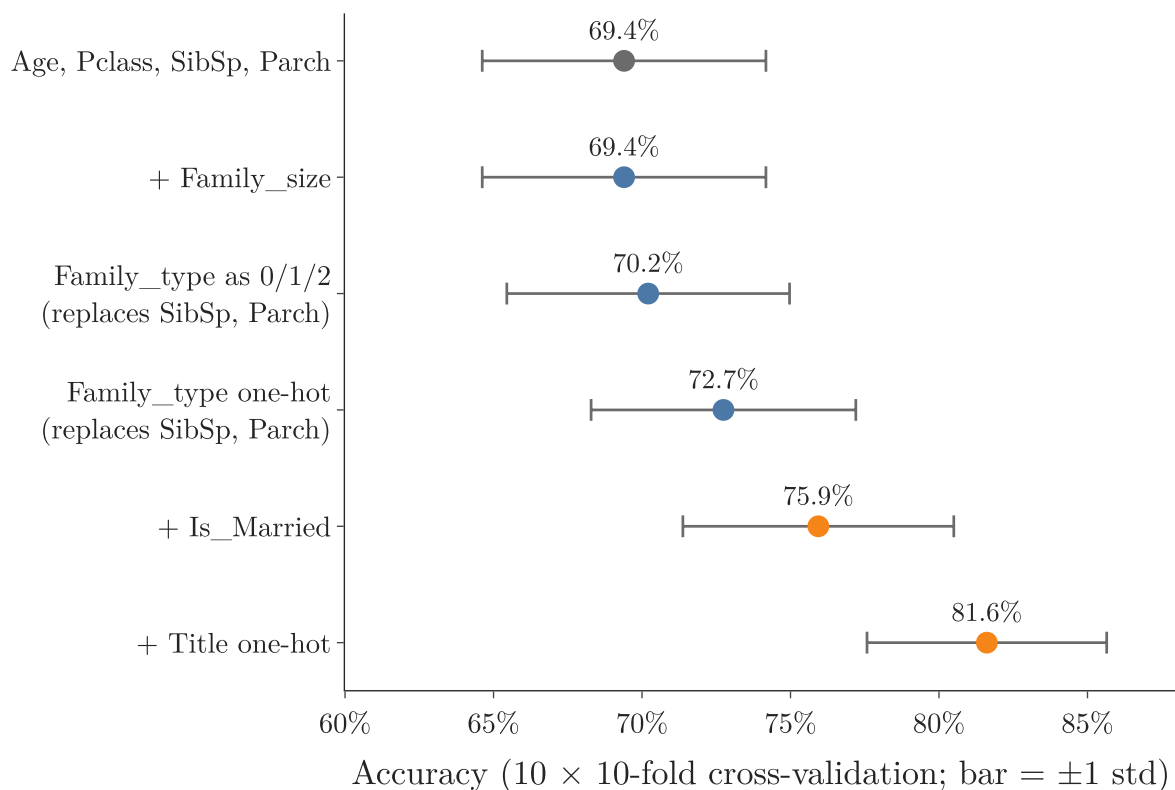


Figure 5: Cross-validated accuracy for each set of columns (grey: baseline; blue: construction; orange: splitting)

Extra: Figure 5 shows the results:

Columns	Accuracy	Change
Age, Pclass, SibSp, Parch (baseline)	69.4%	
baseline + Family_size	69.4%	0.0
Age, Pclass, Family_type as 0/1/2	70.2%	+0.8
Age, Pclass, Family_type one-hot	72.7%	+3.3
baseline + Is_Married	75.9%	+6.5
baseline + Title one-hot	81.6%	+12.2

The grey bars in Figure 5 show the spread of single-fold scores: about 4 to 5 points either way.

Extra: Why `Family_size` adds exactly nothing. Logistic regression gives each column a weight and adds them up, so it can already use $w \cdot (\text{SibSp} + \text{Parch})$ by giving both raw columns the same weight. A new column that is a plain sum of existing ones gives a linear model no new information. Grouping is different: alone, small and large is a bend that no weighted sum of the raw counts can draw.

Extra: Why the family type helps more when one-hot encoded. As one number 0, 1, 2 with a single weight w , the column adds 0, w and $2w$ to the model's score for alone, small and large. These three always go in one direction: if small is above alone ($w > 0$), large must be above small too. But survival goes up and then down (Figure 2). One-hot encoding (Note 27) gives each group its own weight, so the model can follow the up-and-down shape. The 0/1/2 version beats the baseline in only 51 of the 100 folds (29 worse, 20 tied), so its 0.8-point gain is within luck.

Extra: The title is clearly worth keeping: it beats the baseline in 99 of the 100 folds and ties in the last one.

Much of its power is sex: the original columns contain no sex column, and Mr, Mrs and Miss reveal it. Adding the `Sex` column instead gives 80.5%. The rest of the title's edge comes from "Master", which also marks young boys: adding a 0/1 `Master` column to baseline + `Sex` gives 82.9%, even above the title's 81.6% (Notebook, cell after the score table). In the one-hot encoder, titles with fewer than 10 passengers are grouped into one "infrequent" column with `min_frequency=10`, as suggested in Section 7 (scikit-learn User Guide, "Infrequent categories").

Extra: Is building columns before the split data leakage? Family size, family type and title are each computed from one row alone, with a fixed rule, so no information passes from test rows to training rows. Anything that learns from the data, such as the list of common titles in the encoder or a mean used to fill gaps, must be learned inside the pipeline, from training rows only.

9. Summary

	Feature construction	Feature splitting
What it does	builds a new column from existing ones	breaks one column into several
When to use it	domain knowledge suggests a more meaningful column	one cell holds several facts (not tidy)
Titanic example	<code>SibSp + Parch + 1</code> → family size → family type	<code>Name</code> → <code>Title</code>
Other examples	strike rate, economy rate, price per square foot	<code>Cabin</code> → deck and number (Note 33)
Measured gain (logistic regression)	+0.8 points as 0/1/2, +3.3 one-hot	+12.2 points one-hot
pandas tools	column arithmetic, <code>np.select</code>	<code>.str.split</code> , <code>.str[i]</code>

- Feature construction has no formula; it comes from intuition, domain knowledge and experience.
- For every dataset, ask whether a new column could be built that helps the model.
- Tidy data has one observation per row and one atomic value per cell; splitting restores this.
- A split-out column can feed further construction, as with `Is_Married` from `Title`.
- Always score the model before and after a new column, with cross-validation; a gain smaller than the fold-to-fold spread may be luck.
- A plain sum of existing columns adds nothing to a linear model; groups, ratios and split-out facts can.

10. Sources

- Kuhn, M. and Johnson, K. (2019). *Feature Engineering and Selection: A Practical Approach for Predictive Models*, Section 1.1. CRC Press. <https://feat.engineering/>
- Wickham, H. and Grolemund, G. (2017). *R for Data Science*, Section 12.2, “Tidy data”. O’Reilly.
- scikit-learn User Guide. Encoding categorical features, “Infrequent categories”. <https://scikit-learn.org/stable/modules/preprocessing.html>

11. Key terms

Term	Meaning
Family size	<code>SibSp + Parch + 1</code> : the number of people in a passenger’s travelling family
Family type	Family size grouped into alone, small family (2 to 4) and large family (5 or more)

Term	Meaning
Strike rate	A batter's runs per 100 balls faced
Economy rate	A bowler's runs conceded per over
Tidy data	Data with one observation per row and one atomic value per cell
Atomic value	A single piece of information in a cell, not several combined
Feature splitting	Breaking a column that holds several facts into one column per fact
Title	The word before a name, such as Mr, Mrs, Miss or Master
Chained assignment	Selecting part of a DataFrame and then changing that selection in a second step; does nothing in pandas 3